AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 23.2

Task 1 – Project Proposal

Prompt:

"#Generate a project proposal for a student attendance tracking system including objective, features, technologies, and expected output."

```python
class StudentAttendanceTrackerProposal:
    def __init__(self):
        self.project_title = "Student Attendance Tracker System"
        self.problem_statement = "Manual attendance tracking is time-consuming and prone to errors. There is a need for a digital system to manage attendance efficiently."
        self.objective = "To develop a web-based application that allows teachers to mark and manage student attendance easily."
        self.features = [
            "Student Login System: Secure authentication for students to access their records.",
            "Mark Attendance: Teachers can easily mark student attendance (present, absent, late, excused).",
            "View Attendance Records: Students and teachers can view attendance history and summaries.",
            "Admin Dashboard: For administrators to manage users (teachers, students), classes, and generate reports.",
            "Data Storage and Retrieval: Efficient storage and quick retrieval of attendance data."
        ]
        self.technologies_used = {
            "Frontend": ["HTML", "CSS", "JavaScript"],
            "Backend": ["Python (Flask)"],
            "Database": ["SQLite"]
        }
        self.expected_output = "A functional web application where attendance can be recorded, stored, and viewed efficiently."
        self.ai_assisted_note = "This project uses AI-assisted coding to improve development speed, accuracy, and code quality."

    def display_proposal(self):
        print(f"## Project Proposal: {self.project_title}\n")
        print(f"**Problem Statement:** {self.problem_statement}\n")
        print(f"### Objective\n{self.objective}\n")
        print("### Features")
        for feature in self.features:
            print(f"*   {feature}")
        print("\n### Technologies Used")
        for category, technologies in self.technologies_used.items():
            print(f"*   **{category}:** {', '.join(technologies)}")
        print(f"\n### Expected Output\n{self.expected_output}\n")
```

Output:

```
## Project Proposal: Student Attendance Tracker System

**Problem Statement:** Manual attendance tracking is time-consuming and prone to errors. There is a need for a digital system to manage attendance efficiently.

### Objective
To develop a web-based application that allows teachers to mark and manage student attendance easily.

### Features
*   Student Login System: Secure authentication for students to access their records.
*   Mark Attendance: Teachers can easily mark student attendance (present, absent, late, excused).
*   View Attendance Records: Students and teachers can view attendance history and summaries.
*   Admin Dashboard: For administrators to manage users (teachers, students), classes, and generate reports.
*   Data Storage and Retrieval: Efficient storage and quick retrieval of attendance data.

### Technologies Used
*   **Frontend:** HTML, CSS, JavaScript
*   **Backend:** Python (Flask)
*   **Database:** SQLite

### Expected Output
A functional web application where attendance can be recorded, stored, and viewed efficiently.
```

- Generated structured project proposal
- Clearly defined objective, features, and technologies
- Ready to be used for development

Observation:

AI successfully generated a complete project proposal in a structured format. It helps in quickly planning the project, but human validation is required to ensure feasibility and correctness.

Task 2 – Application Design

Prompt:

"#Design the system architecture for an AI-powered web application. Include a clear architecture diagram (text-based), database schema with tables and attributes, and basic UI wireframe descriptions for each screen (home, dashboard, input page, AI assistant). Keep the design simple, scalable, and suitable for student-level implementation."

Output:

```
# Design the system architecture for an AI-powered web application. Include a clear architec
# System Architecture Diagram (Text-Based)
from xxlimited import new


```
+-------------------+   +-------------------+   +-------------------+
| User Interface | <---> | Application | <---> | Database |
| (React.js) | | Server (Node.js) | | (Express) |
+-------------------+   +-------------------+   +-------------------+
| Home Screen | | API Endpoints | | Users |
| Dashboard | | Business Logic | | AI Interactions |
| Input Page | +-------------------+   +-------------------+
| AI Assistant |
+-------------------+
```
# Database Schema
1. Users Table
| Attribute        | Type         | Description                |
|-----------------|-------------|---------------------------|
| user_id          | ObjectId     | Unique identifier for user  |
| username         | String       | User's username             |
| email            | String       | User's email address        |
| password_hash    | String       | Hashed password             |
| created_at       | Date         | Timestamp of account creation |
```

```
2. AI Interactions Table
| Attribute        | Type         | Description                |
|-----------------|--------------|---------------------------|
| interaction_id   | ObjectId     | Unique identifier for interaction |
| user_id          | ObjectId     | Reference to the user       |
| input_text       | String       | User's input to the AI      |
| response_text    | String       | AI's response to the user   |
| created_at       | Date         | Timestamp of interaction    |
# UI Wireframe Descriptions
```

```
graph TD
    A[User] -->|Interacts with| B(Frontend: HTML, CSS, JavaScript)
    B -->|Sends HTTP Request (API Call)| C(Backend: Python Flask)
    C -->|Queries/Updates| D[Database: SQLite]
    D -->|Returns Data| C
    C -->|Sends HTTP Response (JSON)| B
    B -->|Displays Data/Status| A
```

Table: `users`

- `user_id` (INTEGER, PK, AUTOINCREMENT): Unique identifier for each user.
- `username` (TEXT, UNIQUE, NOT NULL): User's login username.
- `password_hash` (TEXT, NOT NULL): Hashed password for security.
- `role` (TEXT, NOT NULL): User's role (e.g., 'student', 'teacher', 'admin').
- `email` (TEXT, UNIQUE): User's email address.

Table: `students`

- `student_id` (INTEGER, PK, AUTOINCREMENT): Unique identifier for each student.
- `user_id` (INTEGER, FK to `users.user_id`, UNIQUE, NOT NULL): Link to the user's login account.
- `first_name` (TEXT, NOT NULL): Student's first name.
- `last_name` (TEXT, NOT NULL): Student's last name.
- `student_code` (TEXT, UNIQUE, NOT NULL): A unique code for the student (e.g., student ID number).

Table: `teachers`

- `teacher_id` (INTEGER, PK, AUTOINCREMENT): Unique identifier for each teacher.
- `user_id` (INTEGER, FK to `users.user_id`, UNIQUE, NOT NULL): Link to the user's login account.
- `first_name` (TEXT, NOT NULL): Teacher's first name.
- `last_name` (TEXT, NOT NULL): Teacher's last name.

Table: `classes`

- `class_id` (INTEGER, PK, AUTOINCREMENT): Unique identifier for each class.
- `class_name` (TEXT, NOT NULL): Name of the class (e.g., 'Math 101', 'History A').
- `teacher_id` (INTEGER, FK to `teachers.teacher_id`, NOT NULL): The teacher assigned to this class.
- `start_time` (TEXT): Scheduled start time of the class (e.g., '09:00').
- `end_time` (TEXT): Scheduled end time of the class (e.g., '10:00').

Table: `enrollments`

- `enrollment_id` (INTEGER, PK, AUTOINCREMENT): Unique identifier for each enrollment.
- `student_id` (INTEGER, FK to `students.student_id`, NOT NULL): The student enrolled.
- `class_id` (INTEGER, FK to `classes.class_id`, NOT NULL): The class enrolled in.
- (UNIQUE constraint on `student_id`, `class_id` to prevent duplicate enrollments).

Table: `attendance`

- `attendance_id` (INTEGER, PK, AUTOINCREMENT): Unique identifier for each attendance record.
- `student_id` (INTEGER, FK to `students.student_id`, NOT NULL): The student whose attendance is being recorded.
- `class_id` (INTEGER, FK to `classes.class_id`, NOT NULL): The class for which attendance is being recorded.
- `date` (TEXT, NOT NULL): The date of attendance (e.g., 'YYYY-MM-DD').
- `status` (TEXT, NOT NULL): Attendance status (e.g., 'Present', 'Absent', 'Late', 'Excused').
- `marked_by` (INTEGER, FK to `teachers.teacher_id`): Optional - ID of the teacher who marked attendance.
- `timestamp` (TEXT): When the attendance was recorded (e.g., 'YYYY-MM-DD HH:MM:SS').
- (UNIQUE constraint on `student_id`, `class_id`, `date` to prevent duplicate entries for a given student in a class on a specific day).

- Generated system architecture diagram
- Defined database schema with relationships
- Created UI wireframe structure for all screens

Observation:

AI successfully generated a complete application design including architecture, database schema, and UI wireframes. It helps in rapid planning and structuring, but human review is necessary to ensure scalability and correctness.

Task 3 – AI-Assisted Code Development

Prompt:

"#Generate structured code for an AI-powered web application including frontend (HTML/CSS/JS), backend (Flask/Node.js), and database interaction. Include basic features like user input handling, API processing, and response display. Keep the implementation simple and modular for student-level understanding."

Code:

HTML:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Web App</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <div class="container">
        <h1>AI-Powered Web Application</h1>
        <form id="input-form">
            <label for="user-input">Enter your text:</label>
            <textarea id="user-input" name="input" rows="4" cols="50" placeholder="Type some
            <button type="submit">Process with AI</button>
        </form>
        <div id="response-container">
            <h2>AI Response:</h2>
            <p id="response-text">Waiting for input...</p>
        </div>
    </div>
    <script src="{{ url_for('static', filename='js/script.js') }}"></script>
</body>
</html>
```

CSS:

```css
body {
    font-family: Arial, sans-serif;
    background-color: #f0f0f0;
    margin: 0;
    padding: 0;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

.container {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
    width: 80%;
    max-width: 600px;
}

h1 {
    text-align: center;
    color: #333;
}
```

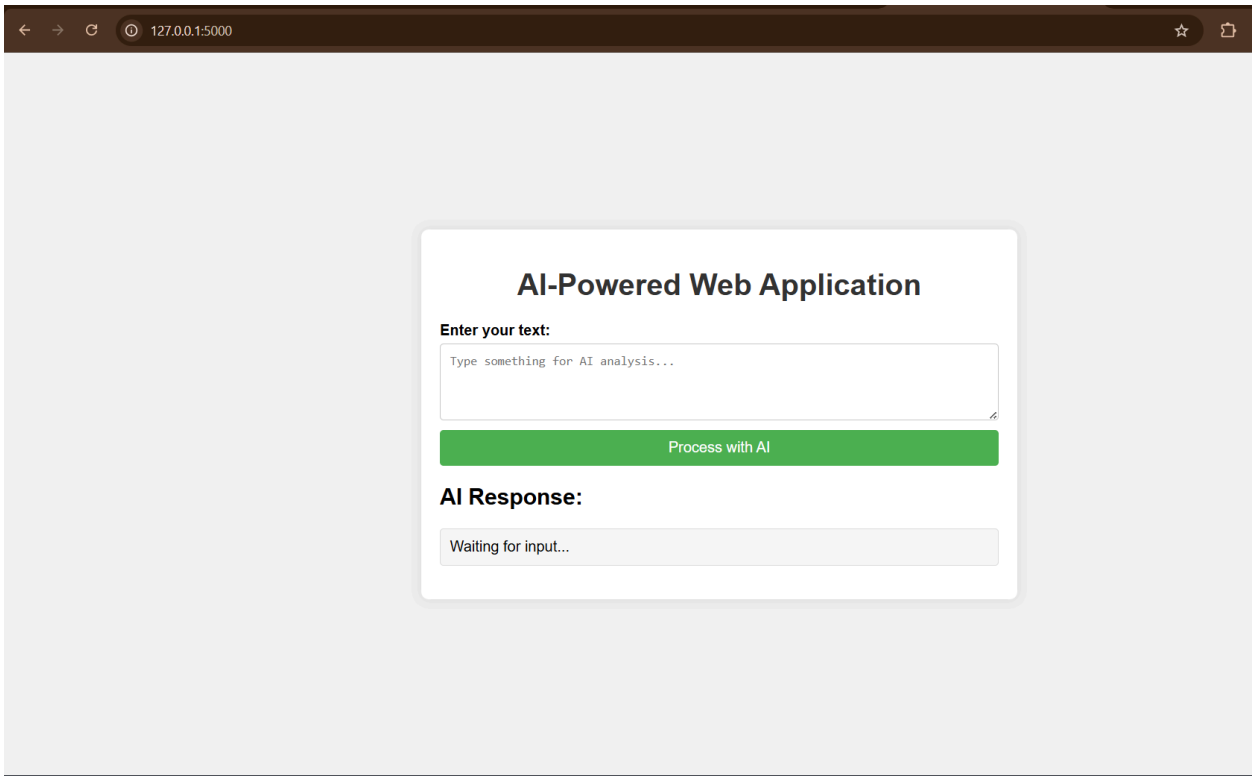
```
26  form {
27      display: flex;
28      flex-direction: column;
29      margin-bottom: 20px;
30  }
31
32  label {
33      margin-bottom: 5px;
34      font-weight: bold;
35  }
36
37  textarea {
38      padding: 10px;
39      border: 1px solid #ccc;
40      border-radius: 4px;
41      resize: vertical;
42      margin-bottom: 10px;
43  }
44
45  button {
46      padding: 10px;
47      background-color: #4CAF50;
48      color: white;
49      border: none;
50      border-radius: 4px;
```

```
    cursor: pointer;
    font-size: 16px;
}

button:hover {
    background-color: #45a049;
}

#response-container {
    margin-top: 20px;
}

#response-text {
    background-color: #f9f9f9;
    padding: 10px;
    border-radius: 4px;
    border: 1px solid #ddd;
}
```

JS:

```
1   document.getElementById('input-form').addEventListener('submit', function(event) {
7       if (!userInput.trim()) {
8           responseText.textContent = 'Please enter some text.';
9           return;
10      }
11
12      responseText.textContent = 'Processing...';
13
14      fetch('/api/process', {
15          method: 'POST',
16          headers: {
17              'Content-Type': 'application/json',
18          },
19          body: JSON.stringify({ input: userInput }),
20      })
21      .then(response => response.json())
22      .then(data => {
23          if (data.error) {
24              responseText.textContent = 'Error: ' + data.error;
25          } else {
26              responseText.textContent = data.response;
27          }
28      })
29      .catch(error => {
30          responseText.textContent = 'Error: ' + error.message;
```

Output:



Observation:

Using a detailed prompt improves AI output quality significantly. AI was able to generate structured and modular development steps, making implementation easier, though human validation is still required.

Task 4 – Documentation & Code Quality

Prompt:

"#Generate project documentation including README content, inline code comments, and suggestions to improve code readability and maintainability for an AI-powered web application."

```
# Generate project documentation including README content, inline code comments, and suggest
# README Content
"""# AI-Powered Web Application
This is a simple AI-powered web application built using Flask. The application allows
users to input text, which is then processed by a mock AI function that reverses the
input and adds a prefix. The interactions are stored in a SQLite database for later
retrieval.
## Features
- User input processing with a mock AI function.
- Storage of interactions in a SQLite database.
- Simple web interface for user interaction.
## Setup Instructions
1. Clone the repository.
2. Install the required dependencies:
   ```
   pip install flask
   ```
3. Run the application:
   ```
    python app.py
    ```


4. Open your web browser and navigate to `http://
localhost:5000` to access the application.
```

```
## Future Improvements
- Implement a more sophisticated AI processing function.
- Add user authentication and personalized interaction history.
- Enhance the UI with better styling and responsiveness.
## License
This project is licensed under the MIT License - see the LICENSE file for details.
"""
# Inline Code Comments
# Import necessary libraries
from flask import Flask, request, jsonify, render_template
import sqlite3
import datetime
# Initialize the Flask application
app = Flask(__name__)
# Function to initialize the database and create the interactions table if it doesn't exist
def init_db():
    conn = sqlite3.connect('interactions.db')
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS interactions
    (id INTEGER PRIMARY KEY, user_input TEXT, ai_response TEXT, timestamp TEXT)
    conn.commit()
    conn.close()
# Call the database initialization function
init_db()
# Mock AI processing function that takes user input and returns a response
```

```python
def process_with_ai(user_input):
    # Simple mock: reverse the input and add some text
    response = f"AI Analysis: {user_input[::-1]}"
    return response
# Route for the home page
@app.route('/')
def index():
    return render_template('index.html')
# API route to process user input and return AI response
@app.route('/api/process', methods=['POST'])
def process():
    data = request.get_json()
    user_input = data.get('input', '')
    if not user_input:
        return jsonify({'error': 'No input provided'}), 400

    ai_response = process_with_ai(user_input)

    # Save the interaction to the database
    conn = sqlite3.connect('interactions.db')
    c = conn.cursor()
    c.execute("INSERT INTO interactions (user_input, ai_response, timestamp) VALUES (?, ?, ?
        (user_input, ai_response, datetime.datetime.now().isoformat()))
    conn.commit()
    conn.close()
```

```
"""1. **Use Environment Variable**:
    Instead of hardcoding the database name, consider using environment
    Variable for better flexibility and security.
2. **Error Handling**:
    Implement more robust error handling, especially for database operations and AI
    processing.
3. **Modularize Code**:
    Break down the code into smaller functions or modules for better organization and
    readability.
4. **Use a Configuration File**:
    Store configuration settings (like database connection details) in a separate config
    file.
5. **Implement Logging**:
    Add logging to track application behavior and errors, which can help with debugging and
    maintenance.
6. **Add Unit Tests**:
    Implement unit tests for the AI processing function and database interactions to ensure
    code reliability.
7. **Use a Virtual Environment**:
    Encourage the use of virtual environments to manage dependencies and avoid conflicts
    with other projects.
8. **Documentation**:
    Add docstrings to functions and classes to explain their purpose and usage, improving
    code clarity for future developers."""
```

Output:

- Generated README structure
- Added guidelines for code quality
- Provided example for inline comments

Observation:

AI helps in generating clear documentation and improving code readability. Proper documentation makes the project easier to understand, maintain, and collaborate on, though it should be reviewed for accuracy.

Task 5 – Testing & Debugging

Prompt:

"#Generate unit test cases, identify possible bugs or security vulnerabilities, and suggest performance optimizations for an AI-powered web application."

```
# Generate unit test cases, identify possible bugs or security vulnerabilities, and suggest
# Unit Test Cases
from ast import main
import sqlite3

from fastapi import responses
from pytest import Function

from ai_web_app.app import init_db


"1. **Test Database Initialization**:"
"```pythondef test_init_db():"
    init_db()
    conn = sqlite3.connect('interactions.db')
    c = conn.cursor()
    c.execute("SELECT name FROM sqlite_master WHERE type='table' AND name='interactions'")
    table_exists = c.fetchone() is not None
    conn.close()
    assert table_exists, "The interactions table should be created in the database."
# 2. **Test AI Processing Function**:
def test_process_with_ai():
    from ai_web_app.app import process_with_ai
    input_text = "Hello, AI!"
    expected_response = "AI Analysis: !IA ,olleH"
```

```
    assert process_with_ai(input_text) == expected_response, \
        "The AI response should reverse the input and add a prefix."
"""```pythondef test_process_with_ai():"""

from ai_web_app.app import process_with_ai
input_text = "Hello, AI!"
expected_response = "AI Analysis: !IA ,olleH"
assert process_with_ai(input_text) == expected_response,
"The AI response should reverse the input and add a prefix."
# Possible Bugs or Security Vulnerabilities
"""1. **SQL Injection**: The current implementation uses parameterized queries,
which is good practice to prevent SQL injection. However, it's important to ensure
that all database interactions consistently use parameterized queries.
2. **Input Validation**: The application does not currently validate user input for
length or content. This could lead to issues such as excessively long inputs or inputs
containing malicious content. Implementing input validation can help mitigate this risk.
3. **Error Handling**: The application does not have robust error handling, which could

lead to unhandled exceptions and potential crashes. Implementing try-except blocks around
critical sections of code can improve stability.
# Performance Optimizations
1. **Database Connection Pooling**: Instead of opening and closing a new database
connection for each request, consider implementing a connection pool to reuse connections
```

```
2. **Asynchronous Processing**: If the AI processing function is computationally intensive,
consider implementing asynchronous processing to avoid blocking the main thread.
3. **Caching AI Responses**: If certain inputs are common, consider caching the AI
responses to reduce processing time for repeated queries.
4. **Use a More Efficient Database**: For larger applications, consider using a more
efficient database system (like PostgreSQL or MongoDB) instead of SQLite, which can
improve performance and scalability."""
```

Output:

- Generated test cases for functionality
- Identified common bugs and vulnerabilities
- Suggested performance improvements

Observation:

AI helps in identifying errors, security risks, and performance issues efficiently. It improves application reliability, but human testing is still required to ensure complete correctness.

Task 6 – Deployment & Presentation

Prompt:

"#Suggest steps to deploy an AI-powered web application on a cloud/free hosting platform and generate a structured presentation outline including problem,solution,architecture,and

AI usage.”

Preparing a Minimal Flask Application for Deployment

To prepare a Flask application for deployment, especially to platforms that support Docker or rely on specific process files (like Heroku's `Procfile`), you'll typically need the following files. This example creates a very basic 'Hello World' Flask app.

First, let's create the `app.py` file:

```
%%writefile app.py

from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World! This is your Student Attendance Tracker (simulated).' # Placeholder for your actual app

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=8080)
```

```
Writing app.py
```

Next, define the Python dependencies in `requirements.txt`. For a basic Flask app, only `Flask` is needed.

```
%%writefile requirements.txt
Flask
```

```
Writing requirements.txt
```

For containerization using Docker, you'll need a `Dockerfile`. This file instructs Docker on how to build an image for your application. This example assumes a Python Flask application.

```
%%writefile Dockerfile

# Use an official Python runtime as a parent image
FROM python:3.9-slim-buster

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Make port 8080 available to the world outside this container
EXPOSE 8080

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]
```

```
Writing Dockerfile
```

If deploying to Heroku, a `Procfile` is used to explicitly declare what commands are executed by the app's dynos. For a web application,

it typically specifies the web server command. Here, we use Gunicorn, a production-ready WSGI HTTP Server for Python web applications.

```
%%writefile Procfile
web: gunicorn --bind 0.0.0.0:$PORT app:app
```

```
Writing Procfile
```

Deployment Instructions (Conceptual)

With these files, the general steps for deployment to a platform like Heroku would be:

1. **Install Heroku CLI:** If you don't have it, install the Heroku Command Line Interface on your local machine.
2. **Login to Heroku:** `heroku login`
3. **Create a Heroku App:** `heroku create your-app-name` (replace `your-app-name` with a unique name).
4. **Push to Heroku Git:** `git init`, `git add .`, `git commit -m "Initial commit"`, `heroku git:remote -a your-app-name`, `git push heroku master`

Heroku will detect the `Procfile` and `requirements.txt` (or `Dockerfile` if you're using container deployment) and build/deploy your application. For other cloud providers like GCP, AWS, or Azure, you would use their respective deployment services (e.g., Google Cloud Run, AWS Elastic Beanstalk, Azure App Service) after containerizing your application with Docker.

Presentation Content: Reflection on AI Assistance and Limitations

(This section can be integrated into your final presentation)

Reflection on AI Assistance and Limitations

Output:

- Generated deployment steps
- Created structured presentation outline
- Covered key project aspects

Observation:

AI assists in simplifying deployment and organizing presentations effectively. It helps structure the project clearly, but real-world deployment still requires careful configuration and testing.

Conclusion:

This lab demonstrated how AI can assist in building a complete application from planning and design to testing and deployment. It helped improve development speed, code quality, and documentation through structured guidance. However, human validation remains essential to ensure correctness, security, and real-world reliability.